Session 25 Homework --- Solution Walkthrough

Status: FINAL (reconciled 2026-11-09)

How to use this walkthrough

This document follows the official Session 25 homework exactly, **HW-01 through HW-42**, in the same order and tier structure as the official step sequence. For every step you can compare what you implemented with the official solution: what the step does, the exact file, the exact code added or changed, why, and what capability exists after it. Two steps (HW-40, HW-42) also carry an explicitly labeled **post-delivery correction** note -- a pre-existing bug in Session 25's own earlier LIVE delivery, found and fixed while implementing that step, and never part of the step's own actual homework delta. This is the official, published solution -- the code shown here is deliberately complete; this is different from the homework assignment itself, where showing the solution is forbidden.

CORE

HW-01 — CORE: install Redux Toolkit

What this step does:

The real `@reduxjs/toolkit` and `react-redux` packages are installed -- this cohort's genuine first real use of Redux in this program.

File: `web/package.json`

Implementation:

Command:

BASH

```
cd web
npm install @reduxjs/toolkit react-redux
```

What changed:

JSON

```
    "@reduxjs/toolkit": "^2.14.0",
    "react-redux": "^9.3.1",
```

Why: Installed now, right before HW-02's own first real use -- the identical real "install right when needed" discipline this program used for Tailwind (Session 24's own LC-01).

After this step: Both real dependencies appear.

HW-02 — CORE: `currentUserSlice`

What this step does:

A real Redux slice exists holding exactly one real thing: the signed-in user or `null`.

File: `web/src/store/currentUserSlice.ts`

Implementation:

TSX

```tsx
import { createSlice, type PayloadAction } from "@reduxjs/toolkit";

export type CurrentUser = {
  id: number;
  name: string;
  email: string;
};

interface CurrentUserState {
  user: CurrentUser | null;
}

const initialState: CurrentUserState = { user: null };

// The ONE real justified Redux use case this app needs -- the real signed-in user, read by
// Client Components (e.g. a navbar) that cannot themselves `await` the real backend. Never
the
// raw JWT (it never reaches the client at all -- see lib/session.ts); never workspace/issue
// data, which stays real server data, fetched fresh, never duplicated here.
const currentUserSlice = createSlice({
  name: "currentUser",
  initialState,
  reducers: {
    setCurrentUser(state, action: PayloadAction<CurrentUser | null>) {
      state.user = action.payload;
    },
  },
});

export const { setCurrentUser } = currentUserSlice.actions;
export default currentUserSlice.reducer;
```

Why: One reducer, `setCurrentUser` -- there is no real "update one field" use case for the current user; it is set once at hydration (HW-05) and cleared on logout (HW-06), always as a whole value.

After this step: Clean exit. Nothing creates a real store instance yet (HW-03).

HW-03 — CORE: the real store factory

What this step does:

A real, typed way to create a fresh Redux store instance exists.

File: `web/src/store/store.ts`

Implementation:

TSX

```tsx
import { configureStore } from "@reduxjs/toolkit";
import currentUserReducer from "./currentUserSlice";

export function makeStore() {
  return configureStore({
    reducer: {
      currentUser: currentUserReducer,
    },
  });
}

export type AppStore = ReturnType<typeof makeStore>;
export type RootState = ReturnType<AppStore["getState"]>;
export type AppDispatch = AppStore["dispatch"];
```

Why: `RootState` / `AppDispatch` are derived FROM `AppStore` with `ReturnType`, never hand-written -- if a second real slice is ever added to the `reducer` map, both types update themselves automatically, with no second place to remember to edit.

After this step: Clean exit. Nothing creates a real store instance yet (HW-04).

HW-04 — CORE: `StoreProvider` -- the real client/server bridge

What this step does:

A real Client Component exists that creates one real store instance per real request and hydrates it with a value a Server Component already fetched.

File: `web/src/store/StoreProvider.tsx`

Implementation:

TSX

```tsx
"use client";

import { useRef } from "react";
import { Provider } from "react-redux";
import { setCurrentUser, type CurrentUser } from "./currentUserSlice";
import { makeStore, type AppStore } from "./store";

// Hydrates the Redux current-user snapshot from a value the real Server Component root
layout
// already fetched (via the session cookie) -- the client never re-fetches or sees the real
// token itself, only the resulting user object.
export function StoreProvider({
```

```
  initialUser,
  children,
}: {
  initialUser: CurrentUser | null;
  children: React.ReactNode;
}) {
  const storeRef = useRef<AppStore | undefined>(undefined);
  if (!storeRef.current) {
    storeRef.current = makeStore();
    storeRef.current.dispatch(setCurrentUser(initialUser));
  }

  return <Provider store={storeRef.current}>{children}</Provider>;
}
```

Why: `useRef`, not `useState` -- the real store must be created exactly once and never trigger a re-render itself; a `ref` is the real tool for "create once, mutate without re-rendering." The `if (!storeRef.current)` guard is what makes it safe to call `makeStore()` on every render without actually creating a second real store each time.

After this step: Clean exit. Nothing renders this component yet.

HW-05 — CORE: the real protected layout fetches and hydrates the current user

What this step does:

Every real page under `(app)` now has a real, hydrated Redux store, carrying the real signed-in user.

File: `web/src/app/(app)/layout.tsx`

Implementation:

TSX

```
import { requireSessionToken } from "@/lib/session";
import { apiFetch } from "@/lib/api";
import { StoreProvider } from "@/store/StoreProvider";
import type { CurrentUser } from "@/store/currentUserSlice";

export default async function AppLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  // Web-adapter check only -- redirects to /login if there's no real session. Every real
  // authorization decision still happens in the real Nest backend on every request this
  // layout's own children go on to make.
  const token = await requireSessionToken();
```

```
  const meRes = await apiFetch("/auth/me", { token });
  const user: CurrentUser = await meRes.json();

  return <StoreProvider initialUser={user}>{children}</StoreProvider>;
}
```

Why: `apiFetch("/auth/me", { token })` is this layout's own first real call that actually sends a token -- the real first live exercise of LC-03's own token support and LC-04's own 401-handling path for an EXPIRED session reaching a real protected page.

After this step: Real `200`. (The real Redux hydration itself is not yet visible -- nothing reads it until HW-05.)

HW-06 — CORE: `logoutAction`

What this step does:

A real way to clear the real session and return to `/login` exists.

File: `web/src/lib/actions/auth.ts`

Implementation:

TSX

```
import { clearSessionToken, setSessionToken } from "@/lib/session";
```

Why: `logoutAction()` takes no arguments at all -- unlike `loginAction` / `registerAction`, a real `<form action={logoutAction}>` with no real form fields needs nothing from the real submission; it is purely "clear the real session, redirect."

After this step: Clean exit. `logoutAction` is not yet called from anywhere (HW-07).

HW-07 — CORE: the real `AccountBar`

What this step does:

A real signed-in user's own name appears, with a real way to log out.

File: `web/src/components/layout/AccountBar.tsx`

Implementation:

TSX

```
"use client";

import Link from "next/link";
import { useSelector } from "react-redux";
import type { RootState } from "@/store/store";
import { logoutAction } from "@/lib/actions/auth";

export function AccountBar() {
```

```
  const user = useSelector((state: RootState) => state.currentUser.user);

  if (!user) {
    return null;
  }

  return (
    <div className="flex items-center justify-end gap-3 border-b border-gray-200 bg-white
px-6 py-2 text-sm dark:border-gray-800 dark:bg-gray-900">
      <Link href="/account" className="text-blue-600 hover:underline dark:text-blue-400">
        {user.name}
      </Link>
      <form action={logoutAction}>
        <button type="submit" className="text-gray-500 hover:underline dark:text-gray-400">
          Log out
        </button>
      </form>
    </div>
  );
}
```

Why: `useSelector((state: RootState) => state.currentUser.user)` -- reading `.user`, not the whole real slice object, is what lets this component only re-render when the real user value itself changes, not on every unrelated store update.

After this step: Clean exit. `AccountBar` is not yet rendered anywhere (HW-08).

HW-08 — CORE: render `AccountBar` on every real protected page

What this step does:

The real account bar actually appears, on every page nested under `(app)`.

File: `web/src/app/(app)/layout.tsx`

Implementation:

TSX

```
import { StoreProvider } from "@/store/StoreProvider";
import type { CurrentUser } from "@/store/currentUserSlice";
import { AccountBar } from "@/components/layout/AccountBar";
```

Why: `<AccountBar />` as a direct child of `<StoreProvider>`, a sibling to `{children}` -- not inside any individual page -- is what makes it appear on every real route under `(app)` without being repeated.

After this step: The real signed-in user's own name appears.

HW-09 — CORE: the real, read-only "My Account" page

What this step does:

A real `/account` page exists, showing the real signed-in user's own name and email.

File: `web/src/app/(app)/account/page.tsx`

Implementation:

TSX

```tsx
import { requireSessionToken } from "@/lib/session";
import { apiFetch } from "@/lib/api";
import { Card } from "@/components/ui/Card";
import type { CurrentUser } from "@/store/currentUserSlice";

export default async function AccountPage() {
  const token = await requireSessionToken();
  const res = await apiFetch("/auth/me", { token });
  const user: CurrentUser = await res.json();

  return (
    <main className="mx-auto flex max-w-sm flex-col gap-4 p-6">
      <Card>
        <h1 className="mb-4 text-xl font-semibold">My Account</h1>
        <p className="text-sm text-gray-500 dark:text-gray-400">Name</p>
        <p className="mb-3">{user.name}</p>
        <p className="text-sm text-gray-500 dark:text-gray-400">Email</p>
        <p>{user.email}</p>
      </Card>
    </main>
  );
}
```

Why: This page calls `requireSessionToken()` / `apiFetch` again itself, rather than threading the layout's own already-fetched `user` down as a prop -- each real Server Component independently gets what it needs, the identical real convention this app already uses everywhere else (e.g. Issue Detail calling its own `apiFetch` rather than receiving issue data as a prop).

After this step: The real signed-in user's own email appears.

HW-10 — CORE: the real `resolveName` helper

What this step does:

A real, reusable helper exists that turns a real user id into a real name, degrading gracefully if the lookup fails -- not yet called from the page.

File: `web/src/app/(app)/workspaces/[id]/projects/[projectId]/issues/[issueId]/page.tsx`

Implementation:

TSX

```
import Link from "next/link";
import { notFound } from "next/navigation";
import { apiFetch } from "@/lib/api";
import { requireSessionToken } from "@/lib/session";
import type { Issue } from "@/lib/types";
import { Card } from "@/components/ui/Card";
import { StatusBadge } from "@/components/ui/Badge";

// Resolving a real id into a real name needs GET /users/:id, which requires auth -- not
// available until this session. A missing/removed user (reporterId/assigneeId still points
at
// a row that no longer resolves) falls back to the raw id rather than breaking the whole
page.
async function resolveName(
  id: number | null,
  token: string,
): Promise<string | null> {
  if (id === null) return null;
  const res = await apiFetch(`/users/${id}`, { token });
  if (!res.ok) return `user #${id}`;
  const user: { name: string } = await res.json();
  return user.name;
}

export default async function IssueDetailPage({
```

Why: The fallback to `` `user #${id}` `` (not a crash, not "unknown") when `GET /users/:id` returns non-ok handles a real, genuine edge case honestly: a referenced user id that no longer resolves (e.g. deleted) should degrade gracefully, not break the whole page.

After this step: Clean exit.

HW-11 — CORE: wire `resolveName` into Issue Detail; CORE complete

What this step does:

Issue Detail shows a real person's own name for reporter/assignee, instead of a raw id -- the real gap every earlier session's own Issue Detail work (Session 23) explicitly disclosed and deferred to here.

File: `web/src/app/(app)/workspaces/[id]/projects/[projectId]/issues/[issueId]/page.tsx`

Implementation:

TSX

```
  const { id, projectId, issueId } = await params;
  const token = await requireSessionToken();
```

```
  const issueRes = await apiFetch(`/issues/${issueId}`);

  if (!issueRes.ok) {
    notFound();
  }

  const issue: Issue = await issueRes.json();
  const [reporterName, assigneeName] = await Promise.all([
    resolveName(issue.reporterId, token),
    resolveName(issue.assigneeId, token),
  ]);
```

Why: `Promise.all`, not two sequential `await`s -- works either way, but needlessly serializes two real, independent network calls when they could run concurrently.

After this step: Real names appear (e.g. "Omar Rahal", "Yara Nassar" for the real seeded issue `WEB-1`), not raw ids.

STRETCH

HW-12 — STRETCH: `createWorkspaceAction` -- the real validation skeleton

What this step does:

A real new export exists that reads a real workspace-creation form and validates it, before it ever calls the real backend.

File: `web/src/lib/actions/workspaces.ts`

Implementation:

TSX

```
"use server";

import { apiFetch } from "@/lib/api";
import { requireSessionToken } from "@/lib/session";

export interface WorkspaceFormState {
  error?: string;
}

export async function createWorkspaceAction(
  _prevState: WorkspaceFormState,
  formData: FormData,
): Promise<WorkspaceFormState> {
  const token = await requireSessionToken();
  const name = String(formData.get("name") ?? "").trim();
```

```
  const slug = String(formData.get("slug") ?? "").trim();
  const description = String(formData.get("description") ?? "").trim();

  if (!name || !slug) {
    return { error: "Name and slug are required." };
  }

  return {};
}
```

Why: `slug` is required here, never derived automatically from `name` -- this item's own disclosed scope is the generic form-action pattern transfer, not a new slugification mechanism.

After this step: Clean exit. Nothing calls the real backend yet (HW-13).

HW-13 — STRETCH: `createWorkspaceAction` -- the real backend call

What this step does:

A real new workspace is actually created against the real backend, with a real duplicate-slug `409` handled inline.

File: `web/src/lib/actions/workspaces.ts`

Implementation:

TSX

```
import { redirect } from "next/navigation";
import { apiFetch } from "@/lib/api";
import { requireSessionToken } from "@/lib/session";
```

Why: A real duplicate-slug `409` is routed to the real inline form error, never a thrown exception/error boundary -- the identical real mistake STRETCH work already warned against for `registerAction`'s own duplicate-email case.

After this step: Clean exit. `createWorkspaceAction` is not yet called from anywhere (HW-14).

HW-14 — STRETCH: `CreateWorkspaceForm` -- the real skeleton

What this step does:

A real form exists, wired to `createWorkspaceAction` through `useActionState`, with no real fields yet.

File: `web/src/app/(app)/workspaces/new/CreateWorkspaceForm.tsx`

Implementation:

TSX

```
"use client";

import { useActionState } from "react";
```

```
import {
  createWorkspaceAction,
  type WorkspaceFormState,
} from "@/lib/actions/workspaces";

const initialState: WorkspaceFormState = {};

export function CreateWorkspaceForm() {
  const [state, formAction, pending] = useActionState(
    createWorkspaceAction,
    initialState,
  );

  return (
    <form action={formAction} className="flex flex-col gap-3">
      {state.error && <p className="text-sm text-red-600">{state.error}</p>}
      <button type="submit" disabled={pending}>
        {pending ? "Creating..." : "Create workspace"}
      </button>
    </form>
  );
}
```

Why: The identical real `useActionState` shape as every other form-action this session built -- proving the pattern transfers again is the point.

After this step: Clean exit. No real fields yet (HW-15/16).

HW-15 — STRETCH: `CreateWorkspaceForm` -- the real name and slug fields

What this step does:

The real form collects a workspace name and slug.

File: `web/src/app/(app)/workspaces/new/CreateWorkspaceForm.tsx`

Implementation:

TSX

```
      {state.error && (
        <p className="text-sm text-red-600 dark:text-red-400">
          {state.error}
        </p>
      )}
      <label className="flex flex-col gap-1 text-sm">
        Name
        <input
          name="name"
          type="text"
```

```
          required
          className="rounded-md border border-gray-300 px-3 py-1 text-sm dark:border-gray-
600 dark:bg-gray-800"
        />
      </label>
      <label className="flex flex-col gap-1 text-sm">
        Slug
        <input
          name="slug"
          type="text"
          required
          placeholder="acme-robotics"
          className="rounded-md border border-gray-300 px-3 py-1 text-sm dark:border-gray-
600 dark:bg-gray-800"
        />
      </label>
      <button type="submit" disabled={pending}>
```

Why: A plain `<input>` for `slug`, not a derived-from-`name` auto-slug -- this item's own disclosed scope is the generic form-action pattern transfer, not a new slugification mechanism.

After this step: Clean exit. The description field and final styling come next (HW-16).

HW-16 — STRETCH: `CreateWorkspaceForm` -- the real description field and styling

What this step does:

The real form collects an optional description, and finally looks like a real styled form.

File: `web/src/app/(app)/workspaces/new/CreateWorkspaceForm.tsx`

Implementation:

TSX

```
      </label>
      <label className="flex flex-col gap-1 text-sm">
        Description (optional)
        <input
          name="description"
          type="text"
          className="rounded-md border border-gray-300 px-3 py-1 text-sm dark:border-gray-
600 dark:bg-gray-800"
        />
      </label>
      <button
        type="submit"
        disabled={pending}
```

```
        className="rounded-md border border-blue-600 bg-blue-600 px-3 py-1 text-sm font-
medium text-white disabled:opacity-50"
      >
        {pending ? "Creating..." : "Create workspace"}
      </button>
```

Why: `description` has no `required` -- the real backend's own `CreateWorkspaceDto` (established earlier) already treats it as optional; the form matches that real contract exactly.

After this step: A real new workspace is created and the real creator becomes its real `owner` (the backend's own existing `WorkspacesService.create` already does this); a real duplicate slug shows the real inline error.

HW-17 — STRETCH: the real `/workspaces/new` page

What this step does:

A real, visitable `/workspaces/new` route exists.

File: `web/src/app/(app)/workspaces/new/page.tsx`

Implementation:

TSX

```tsx
import { Card } from "@/components/ui/Card";
import { CreateWorkspaceForm } from "./CreateWorkspaceForm";

export default function NewWorkspacePage() {
  return (
    <main className="mx-auto flex max-w-sm flex-col gap-4 p-6">
      <Card>
        <h1 className="mb-4 text-xl font-semibold">Create a workspace</h1>
        <CreateWorkspaceForm />
      </Card>
    </main>
  );
}
```

Why: `/workspaces/new` as a literal path segment coexists safely with `/workspaces/[id]` -- Next.js always prefers a literal segment match over a dynamic one at the same level, so a real numeric workspace id can never collide with the literal word `new`.

After this step: `tsc` / `build` both exit clean; real `200`. Nothing yet links to this page (HW-18).

HW-18 — STRETCH: link to the real "New workspace" page

What this step does:

A real signed-in user can actually reach `/workspaces/new` from the real Workspaces list.

File: `web/src/app/(app)/workspaces/page.tsx`

Implementation:

TSX

```
      <div className="flex items-center justify-between">
        <h1 className="text-xl font-semibold">Workspaces</h1>
        <Link
          href="/workspaces/new"
          className="text-sm text-blue-600 hover:underline dark:text-blue-400"
        >
          + New workspace
        </Link>
      </div>
      <div className="flex flex-col gap-3">
```

Why: `Link`, already imported at the top of this real file for the real workspace-card links below -- no new import needed.

After this step: `tsc` / `build` both exit clean; the real link appears and navigates to `/workspaces/new`.

HW-19 — STRETCH: `searchUsersAction`

What this step does:

A real Server Action exists that searches for a real user by name or email, scoped to a real workspace.

File: `web/src/lib/actions/workspace-members.ts`

Implementation:

TSX

```
"use server";

import { revalidatePath } from "next/cache";
import { apiFetch } from "@/lib/api";
import { requireSessionToken } from "@/lib/session";

export interface MemberFormState {
  error?: string;
}

// Called directly from the Client Component below (not bound to a <form>) -- a real
// user-search-as-you-type needs a plain async call, not a form submission.
export async function searchUsersAction(workspaceId: number, q: string) {
  if (q.trim().length < 2) return [];
  const token = await requireSessionToken();
  const res = await apiFetch(
    `/users/search?q=${encodeURIComponent(q)}&workspaceId=${workspaceId}`,
    { token },
```

```
  );
  if (!res.ok) return [];
  return res.json() as Promise<{ id: number; name: string; email: string }[]>;
}
```

Why: `searchUsersAction` returns `[]` on any real non-ok response (never throws) -- a real search box degrading to "no results" is the correct real UX for a transient backend hiccup, not a crashed page.

After this step: Clean exit. Nothing calls this yet (HW-20).

HW-20 — STRETCH: `addMemberAction` -- the real validation skeleton

What this step does:

A real new export exists that validates a real selected user id, before it ever calls the real backend.

File: `web/src/lib/actions/workspace-members.ts`

Implementation:

TSX

```
  if (!res.ok) return [];
  return res.json() as Promise<{ id: number; name: string; email: string }[]>;
}

export async function addMemberAction(
  workspaceId: number,
  _prevState: MemberFormState,
  formData: FormData,
): Promise<MemberFormState> {
  const token = await requireSessionToken();
  const userId = Number(formData.get("userId"));
  const role = String(formData.get("role") ?? "member");

  if (!userId) {
    return { error: "Pick a real user to add first." };
  }

  return {};
}
```

Why: `userId` comes from the real hidden input `AddMemberForm` sets once a user is picked -- a missing/zero value means nothing was actually selected yet, checked before any real network call.

After this step: Clean exit. Nothing calls the real backend yet (HW-21).

HW-21 — STRETCH: `addMemberAction` -- the real backend call

What this step does:

A real workspace admin/owner can add a real searched-for user as a real member.

File: `web/src/lib/actions/workspace-members.ts`

Implementation:

TSX

```
  if (!userId) {
    return { error: "Pick a real user to add first." };
  }

  const res = await apiFetch(`/workspaces/${workspaceId}/members`, {
    method: "POST",
    token,
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ userId, role }),
  });

  if (!res.ok) {
    if (res.status === 409) {
      return { error: "That user is already a member of this workspace." };
    }
    return { error: "Something went wrong. Please try again." };
  }

  revalidatePath(`/workspaces/${workspaceId}/settings`);
  return {};
}
```

Why: `revalidatePath` on the real settings page's own route -- without it, the real newly-added member would not appear until a real full page reload, since Next.js caches the real Server Component render.

After this step: Clean exit. Nothing calls this yet (HW-22).

HW-22 — STRETCH: `AddMemberForm` -- the real search state and branch shape

What this step does:

A real Client Component exists holding the real search-as-you-type state, with its real two-branch shape (picked vs. searching) already in place, each branch still a placeholder.

File: `web/src/app/(app)/workspaces/[id]/settings/AddMemberForm.tsx`

Implementation:

TSX

```
"use client";

import { useState } from "react";
```

```
import { searchUsersAction } from "@/lib/actions/workspace-members";

type SearchResult = { id: number; name: string; email: string };

export function AddMemberForm({ workspaceId }: { workspaceId: number }) {
  const [query, setQuery] = useState("");
  const [results, setResults] = useState<SearchResult[]>([]);
  const [selected, setSelected] = useState<SearchResult | null>(null);

  async function search(value: string) {
    setQuery(value);
    setSelected(null);
    if (value.trim().length < 2) {
      setResults([]);
      return;
    }
    setResults(await searchUsersAction(workspaceId, value));
  }

  return <>{selected ? <p>picked</p> : <p>searching</p>}</>;
}
```

Why: The real `selected ? ... : ...` branch shape is written first, with placeholder content -- once real JSX fills each branch (HW-22/23), the wrapping structure itself never needs to change again.

After this step: Clean exit. No real UI content yet (HW-22/23/24).

HW-23 — STRETCH: `AddMemberForm` -- the real search input

What this step does:

A real search box renders where the "searching" placeholder was.

File: `web/src/app/(app)/workspaces/[id]/settings/AddMemberForm.tsx`

Implementation:

TSX

```
  return (
    <>
      {selected ? (
        <p>picked</p>
      ) : (
        <div className="flex flex-col gap-1">
          <input
            type="text"
            value={query}
            onChange={(e) => search(e.target.value)}
            placeholder="Search by name or email..."
```

```
            className="rounded-md border border-gray-300 px-3 py-1 text-sm dark:border-gray-
600 dark:bg-gray-800"
          />
        </div>
      )}
    </>
  );
}
```

Why: A controlled `<input>` (`value={query}`) -- unlike every other form in this app, this one genuinely needs to react to each keystroke in real time, so it cannot stay uncontrolled.

After this step: Clean exit. The real results dropdown comes next (HW-24).

HW-24 — STRETCH: `AddMemberForm` -- the real results dropdown

What this step does:

Typing in the real search box actually shows real matching users underneath it.

File: `web/src/app/(app)/workspaces/[id]/settings/AddMemberForm.tsx`

Implementation:

TSX

```
            className="rounded-md border border-gray-300 px-3 py-1 text-sm dark:border-gray-
600 dark:bg-gray-800"
          />
          {results.length > 0 && (
            <ul className="flex flex-col gap-1 rounded-md border border-gray-200 p-1
dark:border-gray-700">
              {results.map((user) => (
                <li key={user.id}>
                  <button
                    type="button"
                    className="w-full rounded px-2 py-1 text-left text-sm hover:bg-gray-100
dark:hover:bg-gray-800"
                    onClick={() => {
                      setSelected(user);
                      setResults([]);
                    }}
                  >
                    {user.name} <span className="text-gray-400">{user.email}</span>
                  </button>
                </li>
              ))}
            </ul>
```

```
          )}
        </div>
```

Why: Clicking a result sets `selected` AND clears `results` in the same click -- the dropdown should disappear the instant someone is picked, not linger underneath the next real UI state.

After this step: Clean exit. The real "picked" chip comes next (HW-25).

HW-25 — STRETCH: `AddMemberForm` -- the real picked-user chip

What this step does:

Once a real user is picked, the search box is replaced by a real chip showing who, with a way to change the pick.

File: `web/src/app/(app)/workspaces/[id]/settings/AddMemberForm.tsx`

Implementation:

TSX

```
      {selected ? (
        <div className="flex items-center justify-between gap-2 rounded-md border border-
gray-300 px-3 py-1 text-sm dark:border-gray-600">
          <span>
            <strong>{selected.name}</strong>{" "}
            <span className="text-gray-400">{selected.email}</span>
          </span>
          <button type="button" className="text-xs text-gray-500 hover:underline" onClick=
{() => setSelected(null)}>
            Change
          </button>
        </div>
      ) : (
```

Why: "Change" clears `selected`, returning to the real search branch -- the identical real pattern "clear and re-pick" every other selector in this app would use.

After this step: Clean exit. Nothing submits yet (HW-26).

HW-26 — STRETCH: `AddMemberForm` -- the real submit state

What this step does:

Real state exists to track a role choice, an error, and a pending submit -- not used yet.

File: `web/src/app/(app)/workspaces/[id]/settings/AddMemberForm.tsx`

Implementation:

TSX

```
  const [selected, setSelected] = useState<SearchResult | null>(null);
  const [role, setRole] = useState<"admin" | "member">("member");
  const [error, setError] = useState<string | undefined>();
  const [pending, setPending] = useState(false);

  async function search(value: string) {
```

Why: `role` defaults to `"member"`, the real safer default -- an admin/owner must deliberately change it to `"admin"`, never the other way around.

After this step: Clean exit. Nothing reads these yet (HW-27).

HW-27 — STRETCH: `AddMemberForm` -- the real submit function

What this step does:

A real way exists to submit the real pick -- not yet wired into the returned JSX.

File: `web/src/app/(app)/workspaces/[id]/settings/AddMemberForm.tsx`

Implementation:

TSX

```
import { useState } from "react";
import {
  addMemberAction,
  searchUsersAction,
} from "@/lib/actions/workspace-members";
```

Why: `setPending(false)` runs on every real path (success AND error), before the early `return` on error -- a real submit button must never stay disabled forever after a real failed attempt.

After this step: Clean exit. `submit` is not yet wired into the returned JSX (HW-28).

HW-28 — STRETCH: `AddMemberForm` -- wire the real form; member-add complete

What this step does:

Picking a real user and submitting actually adds them as a real workspace member, with a real role choice and a real inline error on failure.

File: `web/src/app/(app)/workspaces/[id]/settings/AddMemberForm.tsx`

Implementation:

TSX

```
  return (
    <form action={submit} className="flex flex-col gap-2">
```

```
      {error && <p className="text-sm text-red-600 dark:text-red-400">{error}</p>}
      {selected ? (
```

Why: `<input type="hidden" name="userId">`, not threading `selected.id` into `addMemberAction` directly -- the real submission still goes through a real `FormData`, the identical real contract every other Server-Action form in this app uses.

After this step: `npx tsc --noEmit` clean (the settings PAGE itself lands later, but this compiles standalone).

HW-29 — STRETCH: `updateMemberRoleAction` and `removeMemberAction`

What this step does:

A real admin/owner can change a real member's role or remove them.

File: `web/src/lib/actions/workspace-members.ts`

Implementation:

TSX

```
  revalidatePath(`/workspaces/${workspaceId}/settings`);
  return {};
}

export async function updateMemberRoleAction(
  workspaceId: number,
  userId: number,
  role: "admin" | "member",
) {
  const token = await requireSessionToken();

  await apiFetch(`/workspaces/${workspaceId}/members/${userId}`, {
    method: "PATCH",
    token,
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ role }),
  });

  revalidatePath(`/workspaces/${workspaceId}/settings`);
}

export async function removeMemberAction(workspaceId: number, userId: number) {
  const token = await requireSessionToken();

  await apiFetch(`/workspaces/${workspaceId}/members/${userId}`, {
    method: "DELETE",
    token,
  });
```

```
  revalidatePath(`/workspaces/${workspaceId}/settings`);
}
```

Why: Both new actions take `workspaceId` / `userId` as real plain arguments, not a `FormData` -- unlike `addMemberAction`, nothing here is ever submitted from a real `<form>`; each real row's own button/select calls them directly with values it already has in scope.

After this step: Clean exit. Neither action is called from anywhere yet (HW-30).

HW-30 — STRETCH: `MemberRow` -- the real props and state

What this step does:

A real component exists, typed for a single real member row, with no real render yet.

File: `web/src/app/(app)/workspaces/[id]/settings/MemberRow.tsx`

Implementation:

TSX

```tsx
"use client";

import { useState } from "react";
import {
  removeMemberAction,
  updateMemberRoleAction,
} from "@/lib/actions/workspace-members";

export function MemberRow({
  workspaceId,
  userId,
  name,
  role,
  isOwner,
}: {
  workspaceId: number;
  userId: number;
  name: string;
  role: string;
  isOwner: boolean;
}) {
  const [pending, setPending] = useState(false);

  return <div />;
}
```

Why: Five real typed props, no more -- this component receives exactly what it needs to act (ids, display name, current role, owner flag), never the whole real `Workspace` / `Member` object.

After this step: Clean exit. No real behavior yet (HW-29/30).

HW-31 — STRETCH: `MemberRow` -- the real role-change and remove handlers

What this step does:

Real functions exist to change this row's own role or remove it -- not yet wired to any real UI control.

File: `web/src/app/(app)/workspaces/[id]/settings/MemberRow.tsx`

Implementation:

TSX

```tsx
  const [pending, setPending] = useState(false);

  async function changeRole(newRole: "admin" | "member") {
    setPending(true);
    await updateMemberRoleAction(workspaceId, userId, newRole);
    setPending(false);
  }

  async function remove() {
    setPending(true);
    await removeMemberAction(workspaceId, userId);
    setPending(false);
  }

  return <div />;
```

Why: Neither function reads any real return value -- `revalidatePath` inside each real Server Action already handles refreshing the real member list; this component only needs to know when the real call finished, for the real `pending` flag.

After this step: Clean exit. The real render comes next (HW-32).

HW-32 — STRETCH: `MemberRow` -- the real render shape

What this step does:

A real member row shows its own name, with the real owner-vs-editable branch shape already in place -- each branch still a placeholder.

File: `web/src/app/(app)/workspaces/[id]/settings/MemberRow.tsx`

Implementation:

TSX

```
  return (
    <div className="flex items-center justify-between gap-3 rounded-md border border-gray-
200 px-3 py-2 text-sm dark:border-gray-700">
      <span className="font-medium">{name}</span>
      {isOwner ? <span className="text-gray-400">owner</span> : <p>editable</p>}
    </div>
  );
}
```

Why: `isOwner` disables the role/remove controls entirely in the UI, matching the real backend's own server-side guard exactly (`WorkspaceMembersService.requireOwnerIdDoesNotMatch`) -- the UI never offers an action the backend would reject anyway, but the real enforcement still lives server-side, never trusted to the client alone.

After this step: Clean exit. The real editable controls come next (HW-33).

HW-33 — STRETCH: `MemberRow` -- the real editable controls; member CRUD complete

What this step does:

A real non-owner row offers a real role-change select and remove button.

File: `web/src/app/(app)/workspaces/[id]/settings/MemberRow.tsx`

Implementation:

TSX

```
      {isOwner ? (
        <span className="text-gray-400">owner</span>
      ) : (
        <div className="flex items-center gap-2">
          <select
            value={role}
            disabled={pending}
            onChange={(e) => changeRole(e.target.value as "admin" | "member")}
            className="rounded-md border border-gray-300 px-2 py-1 text-xs dark:border-gray-
600 dark:bg-gray-800"
          >
            <option value="member">Member</option>
            <option value="admin">Admin</option>
          </select>
          <button
            type="button"
            disabled={pending}
            onClick={remove}
            className="text-xs text-red-600 hover:underline dark:text-red-400"
```

```
          >
            Remove
          </button>
        </div>
      )}
```

Why: `disabled={pending}` on both controls -- while a real change/remove is in flight, a second real click must not fire a second, overlapping request.

After this step: Clean exit. `MemberRow` is not yet rendered anywhere (HW-38).

HW-34 — STRETCH: Workspace Settings page -- the real workspace fetch

What this step does:

A real page exists that fetches the real workspace itself -- no member list, no render yet.

File: `web/src/app/(app)/workspaces/[id]/settings/page.tsx`

Implementation:

TSX

```tsx
import { notFound } from "next/navigation";
import { apiFetch } from "@/lib/api";
import { requireSessionToken } from "@/lib/session";
import type { Workspace } from "@/lib/types";

export default async function WorkspaceSettingsPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  const workspaceId = Number(id);
  const token = await requireSessionToken();

  const workspaceRes = await apiFetch(`/workspaces/${workspaceId}`, { token });
  if (!workspaceRes.ok) {
    notFound();
  }
  const workspace: Workspace = await workspaceRes.json();

  return <main />;
}
```

Why: `{ token }` is passed explicitly -- unlike the earlier S23/S24 pages (which read real PUBLIC data), `/workspaces/:id` now genuinely requires auth once accessed through this real settings route.

After this step: Clean exit. The real member list fetch comes next (HW-35).

HW-35 — STRETCH: Workspace Settings page -- the real member-list fetch

What this step does:

The real page also fetches the real member list -- no render yet.

File: `web/src/app/(app)/workspaces/[id]/settings/page.tsx`

Implementation:

TSX

```tsx
import type { Workspace } from "@/lib/types";

type Member = {
  userId: number;
  role: string;
  user: { id: number; name: string };
};

export default async function WorkspaceSettingsPage({
```

Why: A real, local `Member` type (not imported from `@/lib/types`) -- this exact shape (`{ userId, role, user: { id, name } }`) is specific to this one real route's own response, not a reusable domain type.

After this step: Clean exit. No real render yet (HW-36/37/38).

HW-36 — STRETCH: Workspace Settings page -- the real header

What this step does:

The real Settings page shows the real workspace name and a real back-link -- the Members card itself still not rendered.

File: `web/src/app/(app)/workspaces/[id]/settings/page.tsx`

Implementation:

TSX

```tsx
import Link from "next/link";
import { notFound } from "next/navigation";
import { apiFetch } from "@/lib/api";
import { requireSessionToken } from "@/lib/session";
import type { Workspace } from "@/lib/types";
```

Why: The back-link and header render unconditionally from real data this page already fetched -- the Members card itself comes next (HW-37), kept as its own real step.

After this step: Clean exit. The real Members card shell comes next (HW-37).

HW-37 — STRETCH: Workspace Settings page -- the real Members card shell

What this step does:

The real Settings page shows a real Members card -- the member list itself still a placeholder.

File: `web/src/app/(app)/workspaces/[id]/settings/page.tsx`

Implementation:

TSX

```tsx
import Link from "next/link";
import { notFound } from "next/navigation";
import { apiFetch } from "@/lib/api";
import { requireSessionToken } from "@/lib/session";
import type { Workspace } from "@/lib/types";
import { Card } from "@/components/ui/Card";
```

Why: The Members card shell renders unconditionally from real data this page already fetched -- only the member LIST itself is still a placeholder, filled in next (HW-38).

After this step: Clean exit. The real member list comes next (HW-38).

HW-38 — STRETCH: Workspace Settings page -- the real member list

What this step does:

The real Settings page renders every real member, reusing `MemberRow`.

File: `web/src/app/(app)/workspaces/[id]/settings/page.tsx`

Implementation:

TSX

```tsx
import { Card } from "@/components/ui/Card";
import { MemberRow } from "./MemberRow";
```

Why: `isOwner={member.userId === workspace.ownerId}` -- computed here, once, from real data this page already fetched, rather than asking `MemberRow` to fetch or guess it itself.

After this step: `tsc` / `build` both exit clean. The real member list renders, the real owner's row shows "owner" with no controls, every other real row offers role-change/remove. The real add-member card comes next (HW-39).

HW-39 — STRETCH: Workspace Settings page -- the real add-member card; settings page complete

What this step does:

The real Settings page also offers the real add-member form.

File: `web/src/app/(app)/workspaces/[id]/settings/page.tsx`

Implementation:

TSX

```
import { Card } from "@/components/ui/Card";
import { AddMemberForm } from "./AddMemberForm";
import { MemberRow } from "./MemberRow";
```

Why: The Add-a-member card is its own `Card`, a sibling of the Members card -- not nested inside it, keeping the real visual grouping consistent with every other multi-section page in this app.

After this step: `tsc` / `build` both exit clean; the real add-member form renders.

HW-40 — STRETCH: link to the real Workspace Settings page

What this step does:

A real admin/owner can actually reach `/workspaces/:id/settings` from the real Workspace Detail page, on desktop.

File: `web/src/app/(app)/workspaces/[id]/page.tsx`

Implementation:

TSX

```
      <div className="flex items-start justify-between">
        <div>
          <h1 className="text-xl font-semibold">{workspace.name}</h1>
          <p className="text-sm text-gray-500 dark:text-gray-400">
            {workspace.description}
          </p>
        </div>
        <Link
          href={`/workspaces/${workspace.id}/settings`}
          className="text-sm text-blue-600 hover:underline dark:text-blue-400"
        >
          Settings
        </Link>
      </div>
```

Why: `Link`, already imported at the top of this real file for the real project-card links below -- no new import needed.

After this step: `tsc` / `build` both exit clean; the real link appears and navigates to the real Settings page. The real mobile drawer (LC-14/15/16/17) does not yet offer this same link (HW-42).

Post-delivery correction encountered while implementing this homework step (not part of this step's own design): the real Session 25 Teamwork Checkpoint had already shipped `workspaces/[id]/page.tsx`'s own Projects heading/list without the `hidden ... md:block` / `hidden flex-col ... md:flex` classes needed to hide the desktop-visible duplicate Projects list once the mobile `WorkspaceSidebar` (LC-14) also lists projects -- a real, pre-existing live-delivery omission, found and fixed while touching this exact file for this step's own Settings link:

TSX

```
      <h2 className="hidden text-sm font-semibold uppercase text-gray-500 dark:text-gray-400
md:block">
        Projects
      </h2>
      <div className="hidden flex-col gap-3 md:flex">
```

This is a correction to Session 25's own earlier LIVE delivery, never part of this homework step's own actual delta.

CHALLENGE

HW-41 — CHALLENGE: why this session split "signed in" across three different places

What this step does:

The student can explain, in their own words, WHY this session never put the real session token anywhere Redux or client JavaScript could read it, and why the Redux `currentUser` slice exists at all if it is not the real source of truth for "is this user actually signed in."

Official reasoning / model answer:

A correct answer, in the student's own words, addresses all three of:

1. **Why the httpOnly cookie, not `localStorage`/Redux, holds the real token.** Any value a Client Component's own JavaScript can read, a malicious script running on the same real page (e.g. via a dependency's own XSS bug) can also read and exfiltrate. An `httpOnly` cookie is invisible to `document.cookie` and every other browser JS API -- only the real Next.js server (via `cookies()`) and the real browser's own network layer (which attaches it to requests automatically) ever see it.
2. **Why `requireSessionToken()` lives in the layout (server), not a client-side check.** A client-side-only check (e.g. "hide the page if Redux has no user") can be bypassed entirely by disabling JavaScript or editing the real response in devtools -- it is a real UX nicety, never real protection. The real protection is `requireSessionToken()` running server-side, before any real page content is ever sent to the browser at all.

3. **Why `currentUserSlice` exists at all, given points 1-2.** It answers a different real question -- not "is this request authenticated" (the backend/layout already decided that before Redux ever runs), but "what does a Client Component, which cannot itself `await` the backend, render for the signed-in user's own name/email right now." It is a real display convenience hydrated FROM an already-authenticated request, never itself an authentication mechanism -- a stale or missing Redux value would show a wrong name, never let an unauthenticated request through.

STRETCH (continued)

HW-42 — STRETCH: the real Settings link in the mobile drawer; HOMEWORK complete

What this step does:

A real admin/owner can reach Workspace Settings from the real mobile drawer too, not desktop only. Declares this session's own real homework-solution checkpoint.

File: `web/src/components/layout/WorkspaceSidebar.tsx`

Implementation:

TSX

```
        <nav className="mt-2 flex flex-col gap-1 rounded-md border border-gray-200 p-2
dark:border-gray-700">
          <Link
            href={`/workspaces/${workspace.id}/settings`}
            onClick={() => setOpen(false)}
            className="rounded px-2 py-1 text-sm text-blue-600 hover:underline dark:text-
blue-400"
          >
            Settings
          </Link>
          <p className="mt-1 px-2 text-xs font-semibold uppercase text-gray-400">
            Projects
          </p>
```

Why: The identical real `onClick={() => setOpen(false)}` every project link in this drawer already uses -- without it, tapping Settings would navigate while the drawer stayed open underneath.

After this step: `tsc` / `build` both exit clean; the real mobile drawer's own markup now includes a Settings link alongside the desktop one HW-34 already added.

Post-delivery correction encountered while implementing this homework step (not part of this step's own design): the real Session 25 Teamwork Checkpoint had already shipped `WorkspaceSidebar.tsx` with a genuine live-coding typo -- `open && (` was written as bare JSX text instead of `{open && (...)}` (missing braces), so the mobile Projects dropdown always rendered, with the literal text `open && (` and `);` visible on the page, and `open` never actually gated anything. Found and fixed while touching this exact file for this step's own Settings link:

TSX

```
      {open && (
        <nav className="mt-2 flex flex-col gap-1 rounded-md border border-gray-200 p-2
dark:border-gray-700">
          ...
        </nav>
      )}
```

This is a correction to Session 25's own earlier LIVE delivery, never part of this homework step's own actual delta.

What the homework asked you to prove

CORE proved the real signed-in user's own identity is genuinely usable client-side (a display name, a working logout, a read-only account page) without ever exposing the real session token to client JavaScript, and that two independent id-to-name lookups (reporter/assignee) resolve concurrently. STRETCH proved the identical generic Server-Action-backed form/list/mutation pattern transfers to two new, unrelated product flows (creating a workspace; managing a workspace's own members) without inventing a second mutation architecture. CHALLENGE proved the student can synthesize, unprompted, why three separate real mechanisms this session built (httpOnly cookie, server-side protected layout, Redux `currentUser`) are each necessary and none of them substitutes for another.

Common mistakes

- Treating the Redux `currentUser` slice as an authentication mechanism -- it is a display snapshot, populated only after the real session check already passed.
- Forgetting `{ token }` on an `apiFetch` call to a guarded route and getting a real `401` that looks like a bug rather than a missing credential.
- Sending `selected.id` directly into a Server Action instead of through the real hidden `<input>` / `FormData` -- breaks the one real submission contract every other form-action in this app uses.
- Nesting the Add-a-member card inside the Members card instead of as its own sibling `Card`.
- Forgetting `onClick={() => setOpen(false)}` on the mobile Settings link -- the drawer stays open underneath the new page.

Exact testing instructions

BASH

```
cd api && PORT=3000 npm run start:dev &
cd web && NEXT_PUBLIC_API_URL=http://localhost:3000 PORT=3001 npm run dev &

# Log in as a seeded user (api/prisma/SEED-AND-LIVE-TEST.md)
TOKEN=$(curl -s -X POST http://localhost:3000/auth/login -H "Content-Type: application/json"
\
  -d '{"email":"omar.admin@b4fboard.dev","password":"Password123!"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['accessToken'])")
```

```
# CORE: hydration, AccountBar, /account, logout
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces | grep -o "Omar Rahal"
curl -s --cookie "session=$TOKEN" http://localhost:3001/account | grep -o
"omar.admin@b4fboard.dev"

# CORE: resolved reporter/assignee names on a real seeded issue
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces/1/projects/1/issues/1 \
  | grep -o "Reporter: [A-Za-z ]*"

# STRETCH: create a workspace, then a duplicate slug
curl -s -X POST http://localhost:3000/workspaces -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"name":"QA","slug":"qa-check"}'
curl -s -w "%{http_code}\n" -X POST http://localhost:3000/workspaces \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"name":"QA2","slug":"qa-check"}'   # expect 409

# STRETCH: Workspace Settings -- member search/add/change-role/remove
curl -s --cookie "session=$TOKEN" http://localhost:3001/workspaces/1/settings | grep -o "Add
a member"
```

To deliberately break something and restore it: comment out the `{ token }` option on the `/auth/me` call in the protected layout -- every page under `(app)` now returns a real `401` and redirects to `/login` even for a valid session, because the real guard sees no `Authorization` header at all. Restore the `{ token }` option to fix it.

What you should understand after studying

- Why the real signed-in user lives in three different places (httpOnly cookie, server-side guard, Redux display snapshot) and what each one alone cannot do.
- Why a Client Component needs a hydrated Redux store instead of `await`ing the backend directly.
- Why `Promise.all` matters for two independent lookups, and when a sequential `await` would still be correct but slower.
- Why the same generic Server-Action form pattern (validate -> call backend -> handle 409 inline -> `revalidatePath` / `redirect`) transfers across completely unrelated features without a new mutation architecture each time.
- Why UI-level owner protection (`isOwner` hiding controls) is never the real security boundary -- the backend's own guard is.